EXPERIMENT 01

Hadamard Gate: Superposition and Measurement

Aim

To implement a Hadamard gate and study measurement outcomes of a single qubit.

Objectives

1. Construct a one-qubit quantum circuit.
2. Create an equal superposition using the H gate.
3. Compare simulated frequencies with theoretical probabilities.

Requirements

Python, Jupyter Notebook, Qiskit, Qiskit Aer, Matplotlib.

Theory

A qubit begins in the computational basis state $|0\rangle$. The Hadamard gate transforms it into an equal-amplitude superposition of $|0\rangle$ and $|1\rangle$. Measurement returns one classical outcome per shot, so repeated shots produce a distribution approaching 50% for each state. This experiment introduces the distinction between quantum amplitudes and observed classical counts.

Key Equation(s)

$H = (1/\sqrt{2})[[1, 1], [1, -1]]$; $H|0\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$; $P(0) = P(1) = 1/2$.

Algorithm / Steps of Execution

1. Import QuantumCircuit, AerSimulator and plotting functions.
2. Create one quantum register and one classical register.
3. Apply the Hadamard gate to the qubit.
4. Measure the qubit into the classical bit.
5. Run the circuit for 1,000 shots and retrieve counts.
6. Plot the histogram and calculate empirical probabilities.

Applications

1. Superposition demonstrations.
2. Quantum randomness education.
3. Circuit simulation and measurement analysis.

Notebook: EXP-01.ipynb

EXPERIMENT 02

Hadamard Circuit Visualization and Probability Estimation

Aim

To visualize a Hadamard circuit and calculate observed output probabilities.

Objectives

1. Draw a quantum circuit with a Hadamard gate.
2. Run repeated measurements in a simulator.
3. Estimate outcome probabilities from measurement counts.

Requirements

Python, Jupyter Notebook, Qiskit, Qiskit Aer, Matplotlib.

Theory

A quantum circuit diagram shows the sequence of operations performed on a qubit. Applying a Hadamard gate to $|0\rangle$ creates the $|+\rangle$ state, whose measurement probabilities are equal in the computational basis. By dividing each observed count by the total number of shots, the experiment estimates these probabilities. Its implementation overlaps substantially with Experiment 01 but emphasizes diagram generation and numerical reporting.

Key Equation(s)

$|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$; $\hat{p}(x) = N(x)/N$; $\hat{p}(0) + \hat{p}(1) = 1$.

Algorithm / Steps of Execution

1. Import circuit, register, simulator and visualization modules.
2. Create a one-qubit circuit with one classical bit.
3. Apply H and add a measurement operation.
4. Render the circuit using the Matplotlib drawer.
5. Simulate 1,000 shots and collect the result counts.
6. Display the histogram and print the estimated probabilities.

Applications

1. Quantum circuit documentation.
2. Statistical measurement exercises.
3. Verification of gate behavior.

Notebook: EXP-02.ipynb

EXPERIMENT 03

Bell-State Generation Using H and CNOT

Aim

To prepare and measure an entangled two-qubit Bell state.

Objectives

1. Prepare a two-qubit circuit.
2. Use Hadamard and CNOT to create entanglement.
3. Inspect correlated measurement outcomes.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

Two qubits initialized in $|00\rangle$ can be entangled by applying a Hadamard gate to the first qubit and a CNOT gate controlled by that qubit. The resulting Bell state is a coherent superposition of $|00\rangle$ and $|11\rangle$. Measurements in the computational basis produce these two outcomes with approximately equal frequency. The circuit illustrates quantum correlation, although a single measurement basis alone does not fully certify entanglement.

Key Equation(s)

$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$; $\text{CNOT}(H \otimes I)|00\rangle = |\Phi^+\rangle$.

Algorithm / Steps of Execution

1. Import QuantumCircuit, simulator and plotting tools.
2. Create a two-qubit circuit with two classical bits.
3. Apply H to qubit 0 and CNOT from qubit 0 to 1.
4. Measure both qubits and draw the circuit.
5. Run the circuit with 1,000 shots.
6. Plot counts and verify that 00 and 11 dominate.

Applications

1. Quantum communication foundations.
2. Entanglement demonstrations.
3. Bell-state preparation for protocols.

Notebook: EXP-03.ipynb

EXPERIMENT 04

Bell-State Simulation and Measurement Histogram

Aim

To execute a Bell-state circuit and examine shot-based outcome statistics.

Objectives

1. Construct a Bell-pair circuit.
2. Compile and simulate the circuit.
3. Analyze the measurement histogram.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

A Bell-pair circuit uses H followed by CNOT to transform two initially unentangled qubits into a maximally entangled state. Running the measured circuit on a shot-based simulator produces classical bit strings and counts. Repeated execution gives approximately half 00 and half 11 in an ideal simulator. This practical revisits the Bell-state preparation of Experiment 03 with explicit transpilation and simulator execution.

Key Equation(s)

$|\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}$; $P(00) = P(11) = 1/2$; $P(01) = P(10) = 0$.

Algorithm / Steps of Execution

1. Create a two-qubit, two-classical-bit circuit.
2. Apply H to the first qubit and CX to the second.
3. Add measurements for both qubits.
4. Initialize AerSimulator and transpile the circuit.
5. Execute 1,024 shots and retrieve counts.
6. Display the circuit and plot the measurement distribution.

Applications

1. Quantum simulator practice.
2. Correlated measurement analysis.
3. Reproducible Bell-state demonstrations.

Notebook: EXP-04.ipynb

EXPERIMENT 05

Three-Qubit Circuit Using CNOT and SWAP

Aim

To demonstrate controlled operations and state exchange in a three-qubit circuit.

Objectives

1. Prepare a three-qubit register.
2. Apply H, CNOT and SWAP in sequence.
3. Interpret the resulting measurement distribution.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

Multi-qubit circuits combine gates to produce correlations and move quantum states between wires. In the supplied implementation, H creates a superposition on qubit 0, CNOT correlates qubits 0 and 1, and SWAP exchanges the states carried by qubits 1 and 2. The final state has correlated computational-basis outcomes across qubits 0 and 2. Measurements and a histogram reveal how gate order changes the resulting bit strings.

Key Equation(s)

$SWAP|a,b\rangle = |b,a\rangle$; $|\psi final\rangle = (|000\rangle + |101\rangle)/\sqrt{2}$ (qubit order $q_2q_1q_0$).

Algorithm / Steps of Execution

1. Create three qubits and three classical bits.
2. Apply H to qubit 0 to form a superposition.
3. Apply CX with control 0 and target 1.
4. Apply SWAP between qubits 1 and 2.
5. Measure all qubits and run 1,024 simulator shots.
6. Inspect the circuit and plot the output counts.

Applications

1. Quantum register manipulation.
2. Multi-qubit circuit design.
3. Quantum information routing concepts.

Notebook: EXP-05.ipynb

EXPERIMENT 06

Deutsch–Jozsa Algorithm: Constant and Balanced Oracles

Aim

To distinguish constant and balanced Boolean functions using Deutsch–Jozsa circuits.

Objectives

1. Construct constant and balanced oracle circuits.
2. Apply phase kickback and interference.
3. Compare measured input-register outcomes.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

The Deutsch–Jozsa algorithm solves a promised problem: a Boolean function is either constant for every input or balanced, returning 0 for exactly half its inputs. It prepares input qubits in superposition and an auxiliary qubit in $|-\rangle$, queries an oracle, and applies Hadamard gates again. The all-zero input-register outcome identifies a constant oracle, while a nonzero outcome identifies a balanced oracle in the ideal algorithm. The supplied notebook implements both cases.

Key Equation(s)

$U_f|x,y\rangle = |x, y \oplus f(x)\rangle$; $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$; constant $\Rightarrow 0^n$; balanced $\Rightarrow$ nonzero.

Algorithm / Steps of Execution

1. Create separate constant and balanced oracle circuits.
2. Prepare the auxiliary qubit in $|1\rangle$ and apply H to all qubits.
3. Implement the constant oracle using identity operations.
4. Implement the balanced oracle using controlled-X gates.
5. Apply H to the input register and measure it.
6. Simulate both circuits and compare the resulting bit strings.

Applications

1. Quantum algorithm fundamentals.
2. Oracle-based problem solving.
3. Quantum interference demonstrations.

Notebook: EXP-06.ipynb

EXPERIMENT 07

Grover Search for a Marked Two-Qubit State

Aim

To implement one iteration of Grover search for the marked state $|11\rangle$.

Objectives

1. Prepare an equal superposition of search states.
2. Mark the target state with a phase oracle.
3. Amplify and measure the target probability.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

Grover search uses amplitude amplification to increase the probability of a marked item in an unstructured search space. For two qubits there are four candidate states. Hadamard gates create a uniform superposition, a controlled-Z oracle changes the sign of the $|11\rangle$ amplitude, and a diffusion operator reflects amplitudes about their average. One ideal Grover iteration is sufficient to obtain the marked state with probability one in this four-item example.

Key Equation(s)

$N = 4$; $|s\rangle = (1/2)\Sigma_x|x\rangle$; $G = (2|s\rangle\langle s| - I)O$; $P(11) = 1$ after one ideal iteration.

Algorithm / Steps of Execution

1. Create a two-qubit circuit and apply H to both qubits.
2. Apply CZ to mark $|11\rangle$ with a phase flip.
3. Apply the diffusion sequence H, X, CZ, X, H.
4. Add measurements to both qubits.
5. Simulate 1,024 shots using AerSimulator.
6. Inspect the histogram for a dominant 11 outcome.

Applications

1. Quantum search education.
2. Amplitude amplification studies.
3. Oracle and diffuser circuit analysis.

Notebook: EXP-07.ipynb

EXPERIMENT 08

Quantum Fourier Transform and Phase Estimation

Aim

To implement QFT and inverse QFT and explore a simplified quantum phase-estimation circuit.

Objectives

1. Construct a three-qubit QFT circuit.
2. Implement inverse QFT operations.
3. Interpret a simplified phase-estimation result.

Requirements

Python, Qiskit, Qiskit Aer, Jupyter Notebook, Matplotlib.

Theory

The quantum Fourier transform maps computational basis states into superpositions whose relative phases encode periodic structure. Its circuit uses Hadamard gates, controlled phase rotations and qubit-order swaps; the inverse reverses these operations. The supplied notebook demonstrates QFT on $|101\rangle$ and a simplified quantum phase-estimation example with a target phase of one quarter. This is a phase-estimation demonstration, not a complete implementation of Shor integer factorization.

Key Equation(s)

$QFT|x\rangle = (1/\sqrt{N})\Sigma_k \exp(2\pi i x k/N)|k\rangle$; $U|u\rangle = \exp(2\pi i\varphi)|u\rangle$; $\varphi = 1/4$.

Algorithm / Steps of Execution

1. Import Qiskit, AerSimulator and the required mathematical utilities.
2. Define a QFT function using H, controlled-phase and SWAP gates.
3. Prepare $|101\rangle$, apply QFT and measure its output.
4. Define the inverse QFT with reversed gate order and angles.
5. Construct the simplified phase-estimation circuit for $\varphi = 1/4$.
6. Simulate both examples and interpret their measurement histograms.

Applications

1. Quantum phase estimation.
2. Quantum algorithm building blocks.
3. Fourier-based quantum computation.

Notebook: EXP-08.ipynb